Designing a simple real time operating system for a ST Micro 32 bit processor

JOSEPH DOBESH, Candidate
FRANK JONES, Advisor

Abstract - In model railroading, the hobbyist has two choices when it comes to controlling a railroad, buying off-the-shelf systems or designing a system from scratch. For someone with the skills, designing a custom system yourself is more satisfying and can be a way to grow your skills. At the heart of such a system is the real time operating system (RTOS) which is the core of this project. It will hereafter be called the UVU RTOS or UTOS. The UTOS starts with a round robin preemptive context switching task manager. Additional features are included in the kernel for the user applications to interact with each other and with the hardware. The UTOS features include mail services, pipelines, Ethernet, mutexing, limited file services and peripheral drivers. This is typical with other RTOS hosted applications, the user applications in this project are built into the kernel during compile time. In this project I will be used model railroad applications to demonstrate the kernel functions. The railroad applications include control of locomotive speed and direction, track switching, and crossing gate control. This project can be easily expanded to include different scheduling methods, kernel functions, more elaborate file systems, extended network features, extended command line functions and more peripheral drivers.

Additional Key Words and Phrases: STM32, RTOS, context switching, model railroad automation

ACM Reference Format:
Joseph Dobesh and Frank Jones. 2025. Designing a simple real time operating system for a ST Micro 32 bit processor. 1, 1 (May 2025), 19 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Model railroading can be a fun and rewarding hobby. Not only can it provide hours of enjoyment, but can provide the hobbyist with opportunities to learn new skills, including electronics and programming. Some hobbyists choose to control their railroad by buying off-the-shelf systems. Many such off-the-shelf systems use the National Model Railroad Association (NMRA) Digital Command Control (DCC) standard[10]. This standard allows different manufacturers to create products that will work with one another. Controllers and modules are strung together on the DDC bus. These systems allow the operator to control everything from the speed of multiple locomotives to sound effects on the layout. Building a railroad using these off-the-shelf systems can be expensive. Designing a custom system yourself can be more satisfying and cost effective.

I have designed and built a railroad control system with a simple real-time operating system (RTOS) at its core. I have called this RTOS the UVU real time operating system, or UTOS. The UTOS differs from typical operating systems in that it has a smaller image footprint, has less features, and is meant to run on embedded

Authors' addresses: Joseph Dobesh, Candidate, 10815159@uvu.edu; Frank Jones, Advisor, FrankJ@uvu.edu.

XXXX-XXXX/2025/5-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

microcontrollers.

There are at least three advantages to running UTOS on your embedded system. One is that the kernel does not have a lot of overhead. This allows more time for the user applications to run. Many times these user applications are controlling hardware that require a near instantanious response from the controller. Any delay could be catastophic. Another advantage of running UTOS is that most microcontrollers have limited memory space, both in program memory and RAM. UTOS has a smaller kernel than other off the shelf operating systems. This makes more efficient use of memory space, both FLASH and RAM. Lastly, UTOS will have features and drivers that do not come standard in a typical PC based operating system, such as Pulse Width Modulation (PWM) and General Purpose Input/Output (GPIO).

One of the defining aspects of UTOS is its round-robin preemptive context switching task schedular. Other additional features are, but not limited to, soft timers, a mail system, FIFOs/pipelines, mutex functionality and serial drivers. In the interest of time and code size a built-in TCP/IP stack was used with HTTP CGI and SSI functionality. This stack was included with the STM32 Cube IDE. Another feature that I have added is command line functionality that can navigate a FAT32 file system on an SD card, run some of the user applications, and test some of the include kernel functions. Most of this project was written in C except for the HTML pages that the operator uses to remotely control the railroad. One page is dedicated to locomotive speed control and direction. One page monitors crossing gate status of the selected crossing gate. And one page monitors the loopback track switch along with the track voltage polarity with the ability to override the switch position. UTOS runs on the ST Micro NUCLEO-F429ZI evaluation board which uses the ST32F429ZI 32-bit ARM Cortex-M4 microcontroller.

Along with the standard operating system features that I have included, I have also added drivers for supporting hardware that is unique to this project. One is a driver for communicating on the Modbus control network. Modbus is a communication protocal for communicating with industrial controls. I have used these controllers to interface the user applications with the model railroad hardware. Modbus is an open-source industrial control communications protocol that can be found at numerous sources on the web.

I have developed three model railroad applications that I have used to demonstrate and validate most of the features of UTOS. The first is the speed control task which controls the speed and direction of the locomotive. This task can be controlled from either an HTML page or the command line prompt. There is also a crossing gate control task which monitors the RR crossings and lowers the crossing gates when a train approaches. The third task is a loopback control that controls polarity and switch direction of a loop when a locomotive is operating within the loop.

2 HARDWARE ARCHITECTURE AND IMPLEMENTATION

This project utilizes the ST Microelectronics Nucleo-F426ZI evaluation board which features the ST Micro ST32F429 ARM Cortex-M4 microcontroller[9]. This evaluation board features many peripherals such as an Ethernet controller, SPI and UART ports, standard I/O and PWM controllers. All of which have been implemented in this project. See Figure 1.

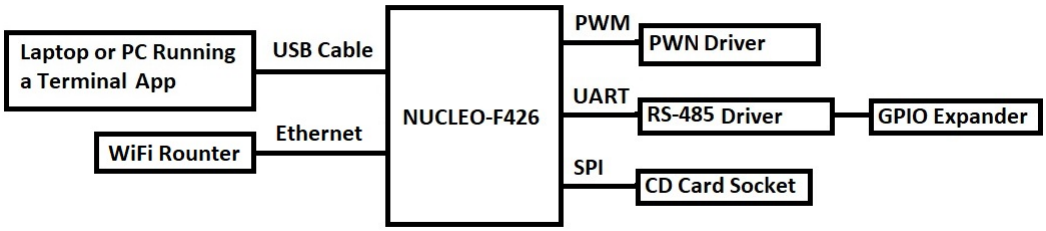

Fig. 1. Upper Level Diagram

There are also other peripheral devices required for controlling the railroad layout, hareafter called the layout. They are listed as follows:

(1) (1) RS-485 Driver, Garosa da22f385-347a-434e-bac3-74f94ecb2d2e
(2) (1) PWM Driver, Anmbest MD114
(3) (1) Modbus Digital I/O expansion module. 16 channels, Eletechsup N4D3E16
(4) (2) Switch Machines, Walthers 942-101
(5) (7) Digital IR Sensors, Iowa Scaled Engineering CKT-IRSENSE
(6) (2) 12VDC 10A DPDT relays, YYUAN MY2NJ
(7) (1) 12VDC 10A Power Supply, Alitove 12V-10A-T
(8) (1) Multiple Output Power Supply (3.3, 5, 12, Adj), NYSUZHOUJI t8uo9nhgpe

The following are figures show simplified block diagrams of each feature used in the project with an explination of its use in the layout.
Figure 2 it shows the power supply arragment. The 12 VDC power supply is needed to supply power to the locomotives running on the layout. It also supplies power to the other low voltage supplies. These supplies power the other devices used in controlling the layout. These power connections are not shown in the diagrams.

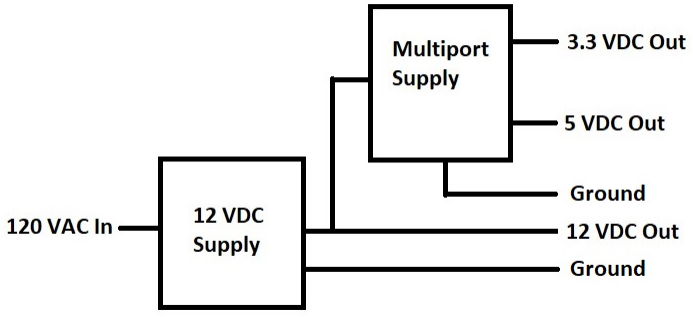

Fig. 2. Power Supplies

Figure 3 shows the wiring diagram for the PWM driver. This driver boosts current and voltage required for controling the speed of the locomotives. An additional diode is supplied to reduce spikes during switching. See Figure 4 for a manufacturers detail of the PWM module.

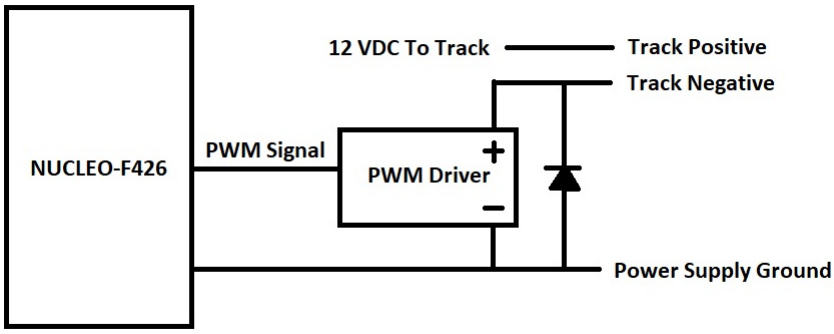

Fig. 3. PWM Circuit

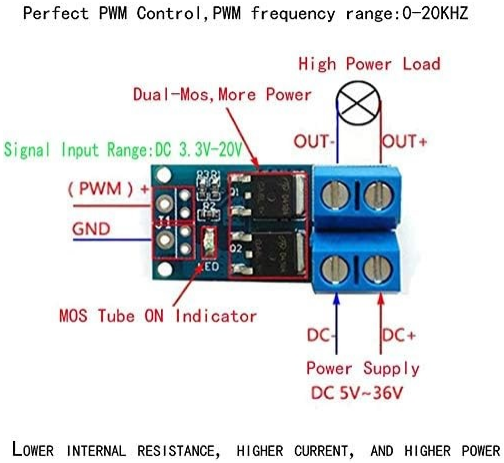

Fig. 4. PWM Module

Figure 5 shows the RS-485 driver. The driver incorporates the Maxim MAX1787 RS-485 transciever[5]. This driver connects to communication lines comming from a Nucleo UART Tx and Rx lines. The DE and /RE control lines are tied together and connected to a GPIO port. This wires the RX-485 driver to work as a half-duplex bus. The DE/RE line switches the driver from Rx mode to Tx mode.

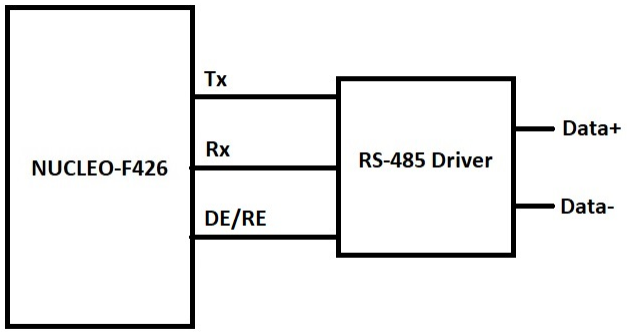

Fig. 5. RS-485 Driver

The last diagram is the most complicated and shows the connections to the Modbus I/O expansion module[2]. This module communicates with the Nucleo through the RS-485 bus. It also provides inputs for the IR sensors on the layout and outputs for the polarity relays and switch motor. The IR sensors detect trains passing over them[3][11]. The relays control the polarity of the voltage going to the locomotives. The switch machines control the direction of the track switches and the crossing gates on the layout on the layout[4]. See Figure 6.

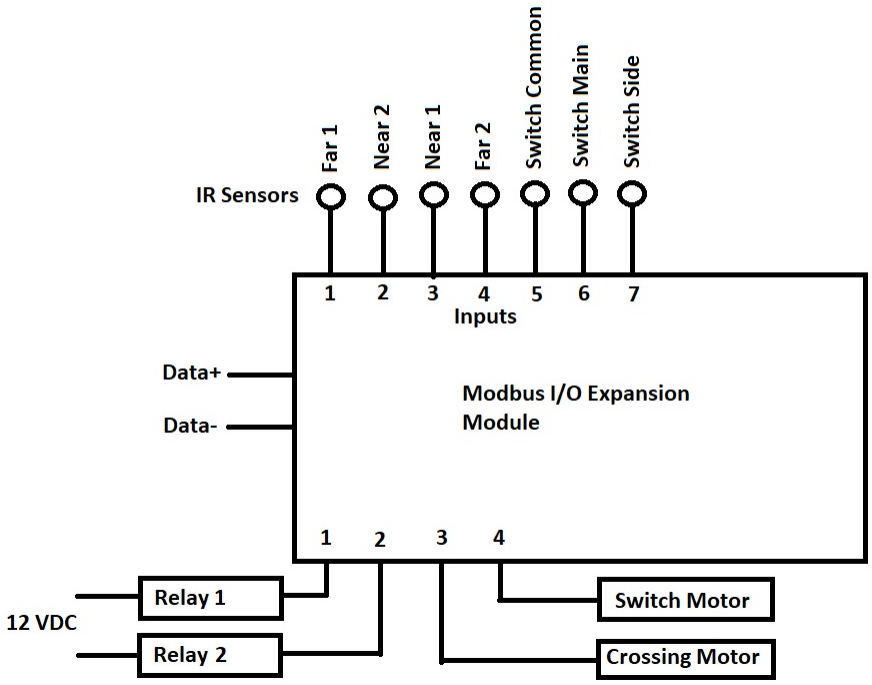

Fig. 6. Modbus I/O Expantion Module

3 SOFTWARE ARCHITECTURE AND IMPLEMENTATION

In this section I will discuss the design and implementation of the Real Time Operating System (RTOS). I will start by explaining the built-in features of the ARM hardware that make creating a preemptive context switching scheduler much simpler. I will then go into the design of the kernel itself and the additional features that make designing applications easier. Finally, I will go into the example model railroad applications themselves.

3.1 Task Manager/Kernel

The core of the real-time operating system (RTOS) is the round-robin preemptive context switching schedular. This scheduler was initially implemented as a simple cooperative task manager. A cooperative task manager relies on tasks that willingly give up CPU time when they are in a waiting or in an idle state. This is not a good architecture because very large tasks can dominate a large amount of CPU time. The programmer would have to make sure to write interruptions periodically throughout the code, which could make it complicated and bloated. Initially, I used this technique to develop and test hardware drivers before moving on to a more complicated kernel. I will not go into further detail on the cooperative switcher because the ultimate goal was to create a preemptive context switcher. The round-robin preemptive context switcher interrupts a task periodically via a timer interrupt. The timer has a constant value and each task is given equal time to run. See Figure 7.

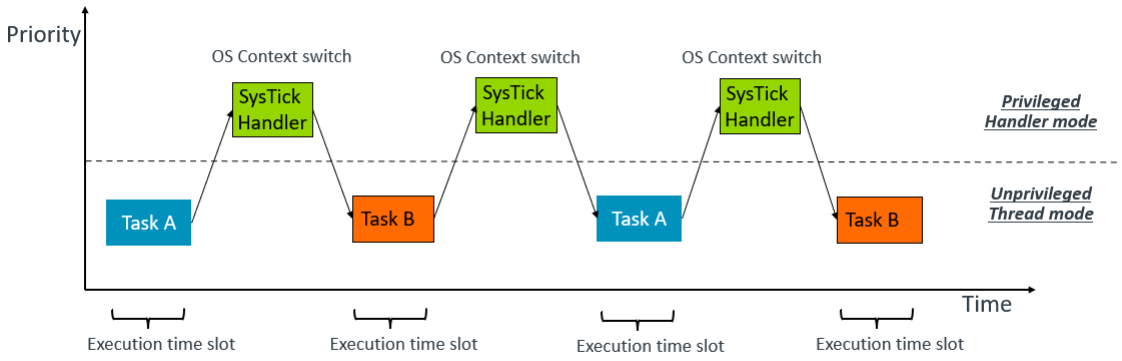

Fig. 7. Preemptive Context Switcher

3.1.1 The ARM Processor. The ARM family of processors have several attributes that make them attractive for building your own RTOS. To begin with, there are 16 dedicated exception vectors that help in creating a scheduler. See Table 1.

Table 1. ARM Exception Table

Exception Number	Exception
1	Reset
2	NMI
3	HardFault
4	MemManage
5	BusFault
6	UsageFault
7-10	Reserved
11	SVCall
12	DebugMonitor
13	Reserved
14	PendSV
15	SysTick

The two exceptions that we will be focusing on are the SysTick and PendSV exceptions. The SysTick exception can be used as a simple tick timer (and I do use this feature to track the blinking time of the Heartbeat LEDs), but it can also be used as a preemptive timer for the context switcher. The number of ticks can be counted and when the switching value is reached, the SysTick routine will set the ICSR bit in the SCB register. This triggers the PendSV exception. After the register is set the switching counter is set back to zero. I made extensive use of the ARM website for this information and examples[13]. See Figure 8.

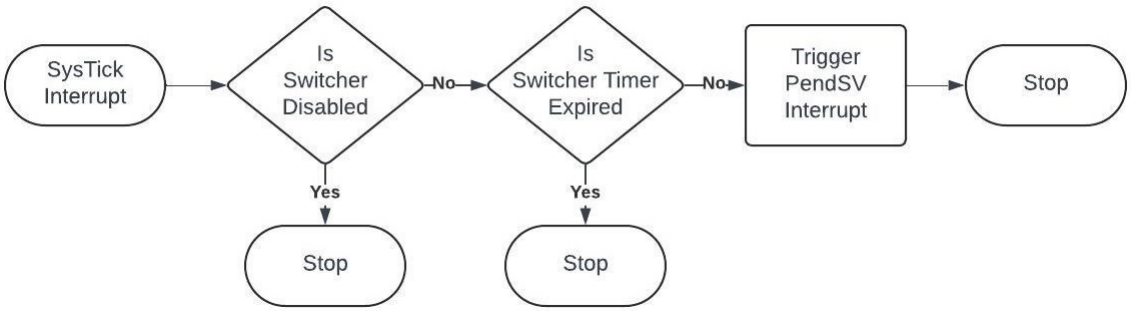

Fig. 8. SysTick Routine

When the PendSV exception is triggered, It begins by saving all the context registers to the Process Control Block (PCB). Each task is assigned a PCB. The PCB is a structure that holds information about the task and also a pointer to the stack where the context registers are saved. More will said about the PCB below. After the context registers are saved, the switching routine in the kernel is called. This routine saves the PCB onto a queue and pulls the next PCB from the queue. Upon returning from this routine, the context register values are copied from the PCB into the contect registers and then the PendSV exception returns. See Figure 9.

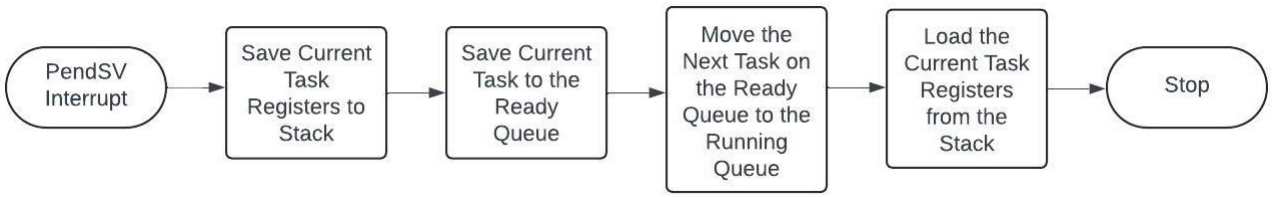

Fig. 9. PendSV Routine

3.1.2 Kernel Init. The STM32 Family of processors come with a vast hardware interface library. I will make use of these libraries in the project. Most of the libraries is initialized in main before the kernel code is called. There is an endless while loop in main where the kernal is called. The kernel call never returns. When the kernel task is called, it first initializes any variables used in the running of the kernel, then it initializes any kernel modules, such as the mail and SD card modules. See Figure 10.

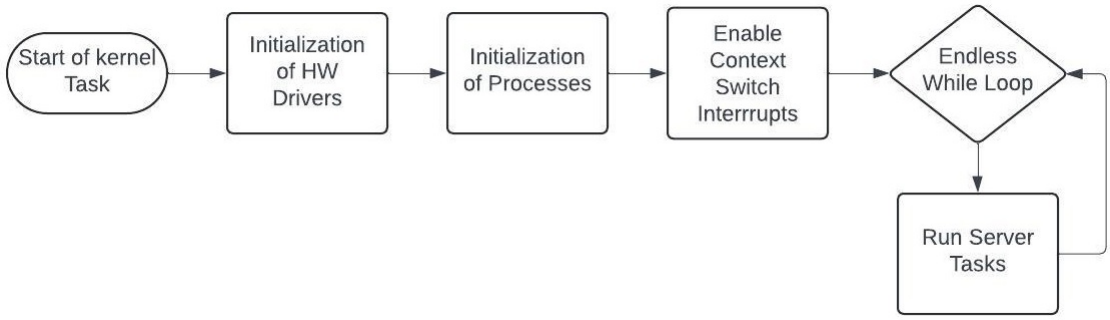

Fig. 10. Kernel Init

After module initialization, the kernel sets up the Ready Queue by allocating a PCB for each task, initiallizing it, and placing it onto the queue. This structure holds critical information for each task, including a pointer to its dedicated stack and the function pointer to the task itself. For the Cortext-M4, the stack pointer must be the first element in the structure and it must be the width of the Stack Pointer (SP) register. In our case, 32 bits.
Once the PCB has been initialized, the dedicated stack is initialized with default values for each saved register. Finally, a pointer to the PCB is pushed into the ready queue. Once each task is initialized and the PCBs are pushed onto the ready queue, the kernel enables the switcher interrupts, and enters its own endless loop.
The first tasks that are pushed onto the ready queue are kernel related tasks. This gives them a chance to run before any of the user tasks. Needless to say, they must be initialized so that if the user tasks need to interact with them, they will function normally.
As mentioned above, when the PendSV exception fires, it saves context registers and then calls the context switching routine. This routine is part of the kernel. The routine starts by pulling the current task PCB pointer value from the current task pointer variable and places it on the back of the pending task queue. It then pulls the next pending task PCB from the queue and copies it to the current task value. It then returns to the PendSV routine.

3.2 RTOS Modules

This section describes the modules that are not part of the context switcher, but are tools that are part of the kernel. These modules make it easier for applications to interact with the kernel, each other and any common resources. There are also modules that are part of the user interface that allow access to the kernel and the applications. I make exstensive use of the book Operating System Concepts for ideas on how to construct these modules[12]. The STM32CubeIDE comes with built in libraries that make it easier to interact with the hardware. I made good use of these libraies[8].

3.2.1 Heartbeat Tasks. The heartbeat tasks are tasks that run as part of the kernel. They are simple tasks that blink the red and blue LEDs every 1 second. This is to indicate to the user that the system is running and not locked up in some way.

3.2.2 Mailbox Task. The Mailbox task is a post office. The module allows a process to send messages to more than one destination. It is protocol independent, which means the messages can be of any protocol to communicate between users. The end users need only know the protocol it will receive from the source. All users need to register a mailbox. If a source mailbox sends a message to an unregistered mailbox, the source will be notified. Every registered mailbox will have its own memory allocation. If the mailbox gets full, the oldest mail will be discarded. The job of the task is to gather mail from source mailboxes and redistribute the mail to the destination mailboxes. See Figure 11. It also has the job of cleaning up any mailboxes that are unregistered. The mailbox is not an efficient tool for steaming data from one task to another, but is usful for sending large messages periodaclaly. See pipeline FIFO below.

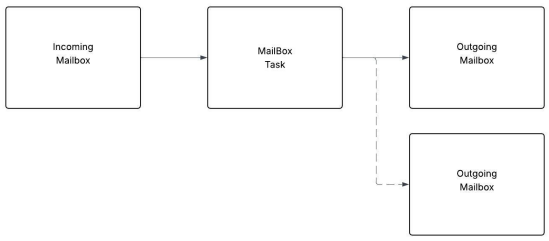

Fig. 11. Mailbox

3.2.3 Command Prompt Task. The Command Prompt task is intended as a testing and debugging tool. The user can type commands to navigate the SD card, set and get RTC date and time or run functional tests of the system. There are four menus to select from in the main menu. Hitting the Enter key or typing help in the main menu displays a help for selecting a sub menu. The command prompt is accessed through the evaluation board's USB cable that also acts as the programming and debugging cable. The user can use a terminal app, like Tera Term, configured as 115200-8-None-1-None to communicate with the controller. Most of the menus are self-explanatory. For more information, see the User's Manual section at the end of this document.

3.2.4 Soft Timer Module. The SoftTimers module is not a task in the sense that was discussed above. It is fully interrupt driven and does not have a task loop. It is meant to supply the different processes access to timers without having to access hardware. The user first registers with the module and if there is an available timer a timer ID is returned. Every time a process desires access the timer from this point, it needs to use its timer ID.

The user has the choice of configuring the timer in two ways; they can configure it as a one-shot timer where the timer does not reset itself when the time expires or it can be set as continuous where the timer resets to a preconfigured time every time it expires. The user also has the choice of passing a callback function pointer to the timer so that when the timer expires, this function is called. The user also has the option of checking the timer manually to see if it has expired. Once the process has finished using the timer, it can release the timer by passing the timer ID into the release function. This is highly recommended as the number of soft timers is limited to 8 pipelines. The soft timers are based on a single hardware timer that fires an interrupt every 10 milliseconds.

3.2.5 Pipeline/FIFO. This module is a merger of pipelines and FIFOs according to the strict definitions. It is more of a pipeline because it does not treat the data buffer as a file. It can pass data within processes or between processes. Also, it can only pass data one way between a single point to another single point. An origin user begins by registering with the pipeline by giving it an unique ID. Part of registering a pipeline involves allocating memory. The destination user need not register, but they do need to know the ID of the sender. The sender passes data on the pipeline and the destination pulls the data off. Once the pipeline is not needed, the sender needs to release the pipeline and return it to the pipeline pool. The receiver should never release the pipeline

3.2.6 Mutex Module. The mutex module is meant to protect resources from being accessed by competing tasks. There is a mutex designator assigned to each resource that may be shared by tasks. Examples are the RS485 port buffers and the SD card. There are three types of blocking functions. The MutexSpinlock method will hold the task in an idle state indefinitely until the block is released by another task. At this point the waiting task will have the block. The MutexWait method will immediately return FALSE if the resource is blocked, otherwise it will return TRUE after grabbing the block. The MutexWaitTime method waits a specified amount of time before returning FALSE if the mutex remains blocked. If the block is released by another task before the timer expires, the block is grabbed by the waiting task and the method returns TRUE. Once a task is done using the resource it must call the MutexRelease method.

3.2.7 Server Stack. Initially, I was going to write my own stack and server to gain more experience with networking. In the interest of time and code size, I decided to instead use the LwIP stack that is included with the STM32 Cube IDE. Along with the stack there are also application layer options that include Common Gateway Interface and Server Side Includes. DHCP was disabled and a static IP address of 192.168.1.111 was assigned. See Figure 12.

Server Side Includes (SSI): This interface allows the HTTPD application layer to interface with the apps running on the controller and receives information from an application and displays it on a HTML page.

Common Gateway Interface (CGI): This interface allows the HTTPD application layer to interface with the apps running on the controller and sends information from a HTML page to the application.

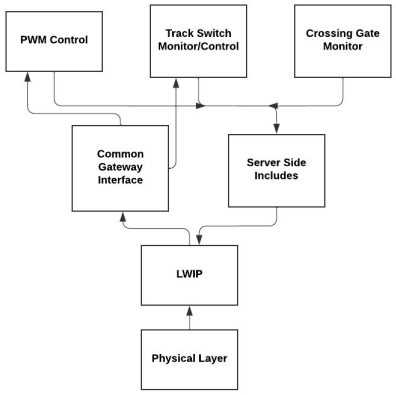

Fig. 12. Server Stack

3.2.8 File System Stack. The file system stack sets up the SD Card for reading files. Example files are in the root directory, which is typical in embedded systems. In the command prompt menu, the user can access the files by using the commands: ls - to list the files and cat - to print the files to the screen[1].

3.2.9 Modbus/RS485 Drivers. These low level hardware drivers create an interface for communicating with the Modbus hardware. The Modbus driver assembles messages that are then loaded into a UART send buffer and are clocked out by the hardware. Incoming messages fire an interrupt which pulls bytes from a UART input buffer and places them into a circular buffer. The Modbus parser then retrieves the message the pertinent data from the message. The serial settings for Modbus are 9600-8-None-1-None.

3.2.10 Real Time Clock (RTC). Support for the built-in RTC is included in the STM32 HAL libraries[8]. It just needed to be set up in the STM32 CubeMX tool. I added the ability to get and set time and date in the command prompt menu.

3.2.11 Watchdog Timer. Support for the built-in Watchdog Timer (WDG) is included in the STM32 HAL libraries[8]. Petting the dog happens in the tick timer interrupt. This way if the interrupts for the context switch ever get disabled by a task (which should never happen), the system will reset.

3.3 User Applications

This project includes three sample user applications. They are discussed in the fallowing subsections.

3.3.1 Locomotive Speed Control. The speed control application is designed to control the speed of the locomotive. It does this by controlling voltage to the tracks using the pulse width modulation (PWM) hardware on the STM32 processor. This signal is fed into a Power MOSFET to step up the voltage and current of the signal. The pulse width is divided into increments of 100, zero being off and 100 being full on. The average voltage can be changed by controlling the duty cycle of the signal. This example demonstrates an application that does not require any built-in RTOS task. The speed can be controlled from the web page or the command prompt.

3.3.2 Crossing Gate Control. The Crossing Gate Control application monitors four sensors that detect the approach of a train. There are two near sensors and two far sensors. The gates are lowered when a far sensor detects a train. See Figure 13.

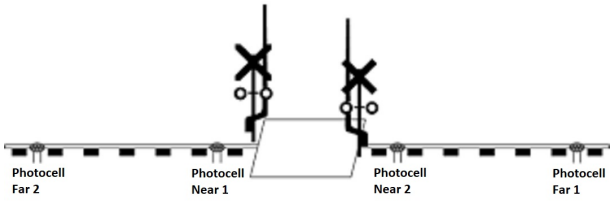

Fig. 13. Crossing Gate Image

The application uses a state machine to progress through the sequence of sensor detections. When the opposite near sensor detects the train, the application moves on to the next state. The gates will remain lowered until both the near and far sensors have cleared. The state machine returns to the idle state when all sensors have cleared. See Figure 14. The sensors are monitored by a Modbus I/O expantion module and must be polled on a regular basis. The gate motor is controlled by the same Modbus I/O expantion module.

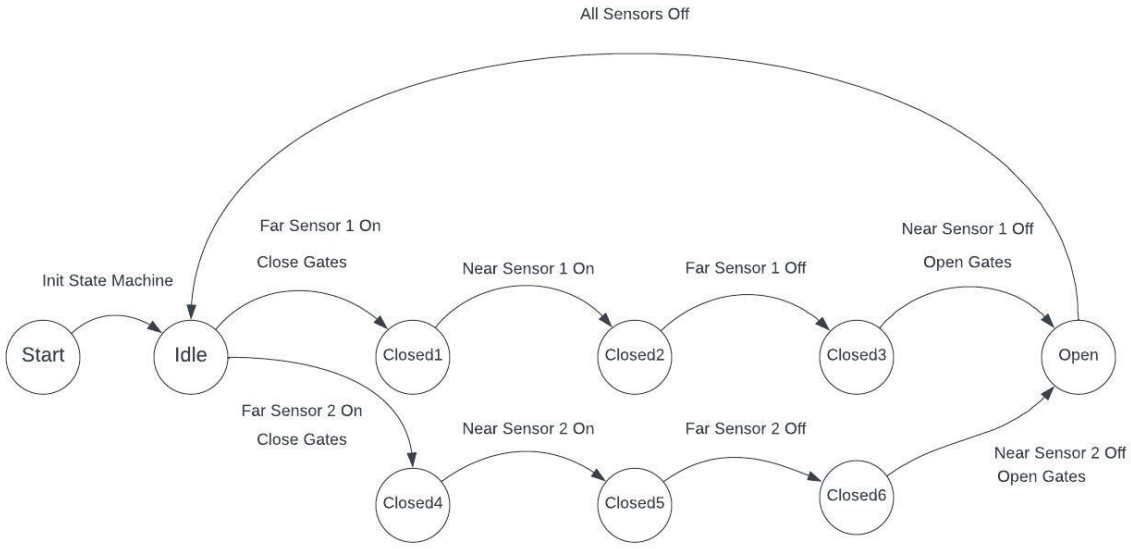

Fig. 14. Gate Control State Machine

3.3.3 Crossover Switch And Polarity Control. The Crossover Switch and Polarity Control application monitors a crossover switch and track voltage polarity to make sure that there are no conditions where a possible derailment could happen. This can happen in two ways. One is when a train tries to pass through a switch where the switch is closed against it. Think of a switch as a 'Y'. When a train is approaching from the base of the 'Y', it has two possibilities for a direction of travel. Thus, there are no chances for derailment. But, if the train is approaching from one of the branches of the 'Y', the switch must be in the position where that branch is selected. If not, the is a very good chance for derailment. There are switches that will give way if this case happens, but this is a chance that I am not willing to take. In this case the controller detects the approach of a train in the branch and if the switch is closed against it, it will open the switch automatically. See Figure 15.

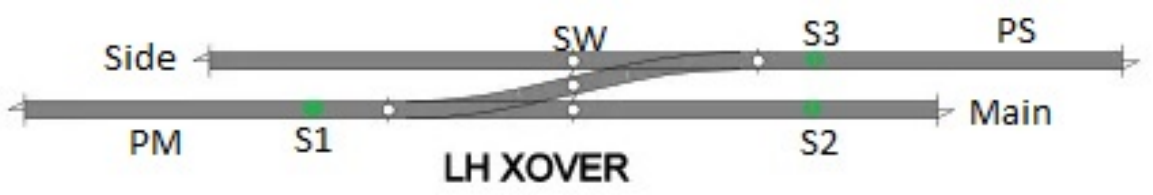

Fig. 15. Crossover Switch

The second derailment condition happens when the train is approaching the crossover switch from the base of the 'Y' and the switch is closed. This means the train is being switched over the adjoining track. If the adjoining track's voltage polarity does not match that of the current track, the locomotive will be suddenly thrown into full reverse. The momentum of the cars will push against the locomotive and cause the train to pancake. The controller monitors the switch position and the voltage polarity of both tracks and makes sure the adjoining track has the same polarity. This truth table in Figure 16 shows the conditions on which polarity and switch position controlled.

Switch Control Truth Table								
	S1=F S2=F S3=F	S1=T S2=F S3=F	S1=T S2=T S3=F	S1=F S2=T S3=F	S1=F S2=T S3=T	S1=F S2=F S3=T	S1=T S2=F S3=T	S1=T S2=T S3=T
SW=O PM=P PS=P	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing
SW=O PM=N PS=P	Do Noting	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing
SW=O PM=N PS=N	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing	Do Nothing (2)	Do Nothing (2)
SW=O PM=P PS=N	Do Nothing	Do Nothing	PS->P	PS->P	PS->P (2)	Do Nothing	Do Nothing (2)	PS->P
SW=C PM=P PS=P	Do Nothing	PS->N	PS->N (1,2)	Do Nothing	Do Nothing (2)	PM->N	PS->N (2)	PS=PM (1,2)
SW=C PM=N PS=P	Do Nothing	Do Nothing	Do Nothing (1,2)	SW->O	Do Nothing (2)	Do Nothing	Do Nothing	Do Nothing (1,2)
SW=C PM=N PS=N	Do Nothing	Do Nothing	Do Nothing (1,2)	SW->O	SW->O	Do Nothing	Do Nothing (1,2)	Do Nothing (1,2)
SW=C PM=P PS=N	Do Nothing	Do Nothing	Do Nothing (1,2)	PS->P	SW->O	PM->N	Do Nothing	Do Nothing (1,2)
	Note 1: Train appears to be strattling the switch							
	Note 2: Possible rolling stock on side or main track							

Fig. 16. Crossover Switch TruthTable

4 PROJECT RESULTS

The Results section is a technical assessment of your project, keeping in mind that it is foremost a learning experience.

(1) What successes did you have?
(2) What failures?
(3) What was completed?
(4) What was planned but not completed?
(5) Looking back, what early choices were good or bad?
(6) What performance issues or bottlenecks exist in your project?

Important things to keep in mind:

(1) An incomplete project is still a success if you learned from it.
(2) Failure is as important a result as success when doing research. The Wright Brothers did not fly on their first attempt, but they did not give up.

The main goal of this project was to create a time interrupt context switching operating system. Although the concepts were easy to understand, implamenting them was a differnt story. For the longest time I was having trouble debugging the context switching routine. I thought it had something to do with the assembly code in the PendSV exception routine. In the end, it turned out to be a optimisation setting in the project settings. Because of this difficluty in debugging, I was forced to learn in more detail how each line of code worked. The final result was a stable context switcher that could switch between multiple tasks without a failure.

As part of the project I created two types tasks, those that tested the the context switcher and those that served a purpose on the model railroad layout. There were two simple test tasks, each blinked an LED. The command prompt could also be conciderd an OS test task, but it was mainly meant to be a way to test hardware.

As for the railroad protion, controlling a single crossing gate or a single crossover switch is easy when you are controlling everything from a central controller. Having said that, things get a lot more complicated when you are controlling numerous instances of this hardware. It hardly makes sense unless your layout is a simple loop with maybe two reverse loops. In this case, using manual controls usually suffices. Unfortunately, this is hardly the case for most model railroad layouts. They tend to be complicated with numerous crossover switches and crossing gates. There is also the possibility that you will be controlling more than one locomotive at a time in different sections of track.

In order to meet the demands of a larger layout, I would make some changes to the current hardware and software design. I would utilize smaller microcontrollers to run as a semi-independent node to control the crossing gates and automate the crossover switches. I would then move the tasks onto these controllers instead of running them on the main comtroller. These smaller controllers would still communicate with the main controller through an RS-485 bus. These semi-autonomous controllers could easily be over-ridden by the main controller if need be. The main controller would also poll these nodes and show the status on a display. I would have to come up with a different communication protocol as Modbus would be insufficient. Instead of running on RS-485, I could use Ethernet connections. That would mean an investment in a large Ethernet switch.

As for as the speed control app, that would remain with the main controller along with an additional feature to isolate different sections of the layout so that more than one locomotive can be controlled at one time. One additional feature not mentioned but is typical in most model railroad layouts is having a physical layout control panel. These panels show the track layout with lights, buttons and switches for controlling track switching and displaying the status of these switches. One other feature that I would add is the control of any animatronics on the layout. It is not uncommon to have a night mode that turns on all the lights in the buildings and streets. There are many more things that can be done with animatronics too numerous to mention here.

4.1 Completed Features

(1) Time Interrupt Context Switching Operating System
(2) Two heartbeat tasks that blinked LEDs
(3) Command Prompt Task
(4) Soft Timer Module
(5) Pipeline-FIFO Module
(6) Mailbox Module/Task
(7) Mutex Module
(8) TCP-IP Stack/Server
(9) PWM Locomotive Speed Control
(10) Crossing Gate Control Task
(11) Track Switch Control Task

4.2 Deleted Features

The following features, that were mentioned in the proposal, were removed from the project because they were deemed unnecessary to demonstrate the RTOS, they did not add value to the railroad applications, or they were redundant. Meaning other features did a better job.

(1) Keypad - Most anything that can be entered on the keypad, can be done from the command prompt or on the web page.
(2) LED Character Display - Most anything that can be displayed on the LED Display, can be displayed from the command prompt or on the web page.
(3) Health Monitor - Temperature and humidity sensors do very littler to demonstrate the operation of the RTOS and are not needed on a model railroad layout.
(4) Push Buttons - Most anything that can be done by a push button, can be done from the command prompt or on the web page.

5 CONCLUSION

The conclusion should address the following items:

(1) Summarize and synthesize your learning and effort on the project.
(2) What is your personal view on the success or failure of the project?
(3) What did you learn in creating the project?
(4) What would have been helpful to know before starting the project?
(5) What was your most important results and conclusions?
(6) What are potential next steps for the project?

The rest of the paper needs be factual and objective in tone, but in the conclusion you are free to express your opinions, beliefs and feelings.

I would say that the most ejoyable portion of this project was learning about and implementing the operating system. The features in the ARM family of processors make the job easier and there are a plethera of web sites and example code out there to learn from. I have taken OS classes before, but the projects were simulated on a PC. I would say that writing an OS for a non PC microprocessor takes the learnig experience to the next level. Along with writing the basic switching routine, writing modules as part of the OS has also helped me understand how these features work as well and has given me insight into which modules work best in certain situations. If I was to do this project over again, I think I would not choose to do the crossing gate and track switch task as examples of tasks to demonstrait the operating system. They reliy too much on hardware and this is a software

project. If I was to continue working on this project, there are many other examples of features that could be added instead that are not as hardware relient. For example:

(1) Adding priorities to the context switcher
(2) Passing parameters to tasks
(3) Event Handling
(4) Expanding the FAT file system
(5) Adding different file systems
(6) Expanding Server Options
(7) Designing a TCP/IP stack from scratch
(8) Logging
(9) LCD display
(10) Keypad
(11) Port to a different MCU
(12) Interprocessor Communications
(13) Network communications between microcontrollers

6 FIGURES AND CAPTIONS

List of Figures

7 REFERENCES

REFERENCES

[1] Jan Axelson. 2006. *USB Mass Storage* (1st ed.). Lakeview Research LLC. www.janaxelson.com (book).

[2] Eletechsup. [n.d.]. *N4D3E16 DC 12V 24V 16 Input 16 Output RS485 Remote Control Switch PLC IO expansion Board 03 06 16 Modbus RTU Module.* Retrieved May 7, 2025 from https://eletechsup.com/products/dc-12v-24v-16-input-16-output-rs485-remote-control-switch-plc-io-expansion-board-03-06-16-modbus-rtu-module
[3] Iowa Scaled Engineering. [n.d.]. *Train Spotter Rock Solid Train Detection, Reflective Infared Proximity Sensor.* Iowa Scale Engineering, Iowa, USA. https://www.iascaled.com/docs/ckt-irsense/ckt-irsense-manual.pdf
[4] Walthers Inc. 2018. *Walthers Control System, Switch Machine Reference Guide.* Walthers Inc, Milwaukee, WI. https://s3.amazonaws.com/aws.walthers.com/942-101+Switch+Machine+Instruction+Sheet+and+Advance+Control+Manual.pdf
[5] Maxim Integrated. 2014. *Low Power, Slew-Rate-Limited RS-485/RS-422 Transceivers.* Maxim Integrated, San Jose, CA. https://www.analog.com/media/en/technical-documentation/data-sheets/MAX1487-MAX491.pdf
[6] ST Microelectronics. [n.d.]. *ST Microelectronics website.* Retrieved May 7, 2025 from https://www.st.com
[7] ST Microelectronics. 2016. *Getting started with STM32 Nucleo board software development tools.* ST Microelectronics, France. https://www.st.com/resource/en/user_manual/um1727-getting-started-with-stm32-nucleo-board-software-development-tools-stmicroelectronics.pdf
[8] ST Microelectronics. 2023. *Description of STM32F4 HAL and low-layer drivers.* ST Microelectronics, France. https://www.st.com/resource/en/user_manual/um1725-description-of-stm32f4-hal-and-lowlayer-drivers-stmicroelectronics.pdf
[9] ST Microelectronics. 2023. *User Manual, STM32 Nucleo-144 Boards (MB1137).* ST Microelectronics, France. https://www.st.com/resource/en/user_manual/um1727-getting-started-with-stm32-nucleo-board-software-development-tools-stmicroelectronics.pdf
[10] National Model Railroad Assoc. (NMRA). 2021. *Electrical Standards for Digital Command Control (DCC).* NMRA, Soddy Daisy, TN. https://www.nmra.org/sites/default/files/standards/sandrp/DCC/S/s-9.1_electrical_standards_for_digital_command_control_2021.pdf
[11] Nathan Holmes/Micheal Petersen. [n.d.]. *Reflective IR Model Railroad Sensor (V4), Schematic.* Iowa Scale Engineering, Iowa, USA. https://github.com/IowaScaledEngineering/ckt-irsense/raw/master/pg/ckt-irsense-panel-v4.1-fd210cc/ckt-irsense.pdf
[12] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. 2013. *Operating System Concepts* (9th. ed.). Wiley, Hoboken, NJ.
[13] ARM Technologies. [n.d.]. *Official ARM Website.* Retrieved May 7, 2025 from https://www.arm.com

A INSTALLATION MANUAL

To begin, it will be nessisary to install ST Micro development software onto a laptop or desk top computer. The tools that were used are STM32CubeMX and and STM32CubeIDE. You can find these tools on the ST MIcro website [6]. STM32CudeMX should be installed first. Follow the installation instructions on the website.
Start STM32CubeIDE and then open the GradProject project. Compile the project by selecting Project in the menu bar and then clicking on Build All. Next, connect the Nucleo board to your computer via a USB cable. The connector ID the Nucleo board is CN1. Power to the Nucleo board is supplied from the computer.
It will also be necessary to install a terminal on your comupter like TeraTerm or Putty. You will be communicating with applications on the platform through the USB port. The UART settings are 115200,8,1,None.
Next, from the menu bar select Run and then click Debug. This will load the firmware and reset the Nucleo board. For more information, see "Getting started with STM32 Nucleo board software development tools" [7].

B USER'S MANUAL

COMMAND LINE INTERFACE, WEB PAGE, SUBSECTIONS,

This section is meant to familiarize a new operator with the model railroad control system outlined in this paper. Usually, the firmware would have already been installed on the platform and the hardware mounted and wired. If the operator is starting from scratch, see the Hardware Section and Installation Instructions above.

B.1 Using The Command Prompt

The Command Prompt started out as a debugging and testing tool, but there are also other options that alow the user to run the railroad manualy. To use the Command Prompt, you should already have the Nucleo board connected to a computer via the USB cable and have a terminal program running. See Installation Maunal above. When the Nucleo board is powered on, it should show the UTOS> prompt. To see the main menu press the Enter

key. The Main Menu is shown in the tabel below.
From the main menu the user can select submenus that have been divided up into catigories for testing and

Table 2. Main Menu

help -	Displays The Main Menu
run -	Sends you to the railroad manual run menu
sd -	Mounts the SD Card
rtc -	RTC Menu
test -	Sends you to the driver test menu

operating the railroad. Entering 'help' just displays the main menu again. Entering 'run' sends the user to the railroad manual apps submenu. Entering 'sd' mounts the SD card (if one is installed) and diplays the SD> prompt. Entering 'rtc' brings the user to the RTC submenu. Entering 'test' brings the user to the test submenu. Entering anything else just redisplays the main menu.
Once in the run menu, the user can select which aspects of the railroad they want to control. Selecting '0' brings the user back to the main menu.
To run the speed control, enter '1'. The app will display a number. This number is the current setting of the

Table 3. Run Menu

Return To Main Menu	- 0
Run Speed Control	- 1
Run Crossing Control	- 2
Run Track Switch Control	- 3

speed control. To increase speed, press the '>' key. To decrease speed, press the '<' key. The new setting will be displayed on the screen. The numbers indicate the pulse width of the PWN signal by percentage. The range of the percentage is from 0 to 100 percent. To exit the speed control app, press '0'.

Table 4. SD Card Menu

cd /	- Return To Main Menu
ls	- List the files i the directory
cat <file>	- Display the contents of the file

Table 5. RTC Menu

Return To Main Menu	- 0
Get RTC Time	- 1
Get RTC Date	- 2
Set RTC Time	- 3
Set RTC Date	- 4

Table 6. Test Menu

Return To Main Menu	- 0
Check Boot Sector	- 1
Check RS-485	- 2
Check Modbus	- 3

B.2 Using The Webserver

The control system is mostly fully automated. The different applications can be controlled or monitored by selecting the right page from the Railroad home page. The Speed Control page allows you to control the speed of the locomotive. The up and down arrows increment or decrement the speed by one percentage. The indicator box displays the current value. The crossing gates work with no intervention from the operator. You can monitor the sensor and crossing gate status on the Crossing Gate page by selecting the crossing in the drop-down box. For loopback or crossover switching, The Loopback page allows the user to open or close the loopback switch. It some cases the manual control is overridden when a critical situation occurs, such as when a train is passing through the switch. I all other cases the switching and polarity is controlled automatically.